```
#KNN Classification Project- Iris Dataset

#Importing libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score,recall_score, f1_score, confusion_matrix, classification_report
```

```
#Loading Dataset
iris =load_iris()
X= iris.data
y =iris.target
```

```
# Creating DataFrame
df = pd.DataFrame(X, columns=iris.feature_names)
df['target'] = y
print("First 5 rows:")
print (df.head())
```

```
First 5 rows:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  target
0                5.1               3.5  ...               0.2       0
1                4.9               3.0  ...               0.2       0
2                4.7               3.2  ...               0.2       0
3                4.6               3.1  ...               0.2       0
4                5.0               3.6  ...               0.2       0

[5 rows x 5 columns]
```

```
# checking missing values
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64
```

```
#spliting data (80/20)
X_train, X_test,y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
print("\nTraining Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)
```

```
Training Shape: (120, 4)
Testing Shape: (30, 4)
```

```
# scale features
scaler = StandardScaler()
X_train_scaled =scaler.fit_transform(X_train)
X_test_scaled =scaler.transform(X_test)
```

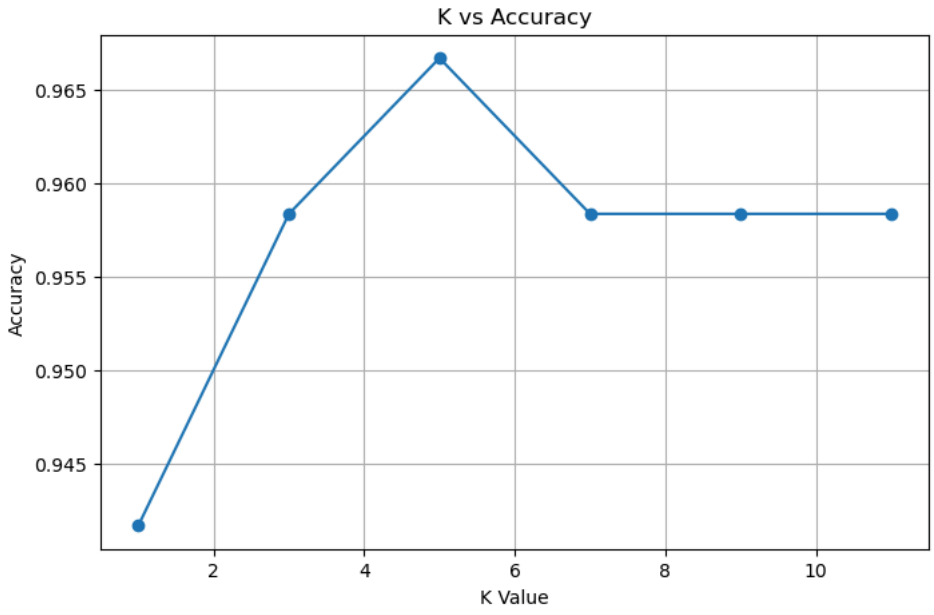
```
# train and test different K values
k_values = [1, 3, 5, 7, 9, 11]
cv_scores =[]

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
```

```
    scores = cross_val_score(
        knn,
        X_train_scaled,
        y_train,
        cv=5,
        scoring='accuracy'
    )
    cv_scores.append(scores.mean())
```

```
#displaying result
results = pd.DataFrame({
    'K': k_values,
    'Accuracy': cv_scores
})
print("\nCross Validation Results:")
print(results)
```

```
Cross Validation Results:
    K  Accuracy
0   1  0.941667
1   3  0.958333
2   5  0.966667
3   7  0.958333
4   9  0.958333
5  11  0.958333
```

```
# plotting K vs Accuracy
plt.figure(figsize=(8,5))
plt.plot(k_values, cv_scores, marker='o')
plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("K vs Accuracy")
plt.grid(True)
plt.show()
```




```
# finding best K
best_k = k_values[np.argmax(cv_scores)]
print("\nBest K:", best_k)
```

```
Best K: 5
```

```
# train final model
final_model = KNeighborsClassifier(n_neighbors=best_k)
final_model.fit(X_train_scaled, y_train)
```

```
▾ KNeighborsClassifier
▶ Parameters
```

```
#Predictions
y_pred = final_model.predict(X_test_scaled)
```

```
#Evaluation Metrics
accuracy = accuracy_score(y_test,y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
print("\nModel Evaluation")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

```
Model Evaluation
Accuracy : 0.9333333333333333
Precision: 0.9444444444444445
Recall   : 0.9333333333333333
F1 Score : 0.9326599326599326
```

```
#Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]
```

```
#Testing new sample data
new_samples = np.array([
    [5.1, 3.5, 1.4, 0.2],
    [6.0, 2.9, 4.5, 1.5],
    [6.7, 3.1, 5.6, 2.4]
])
new_sample_scaled = scaler.transform(new_samples)

predictions = final_model.predict(new_sample_scaled)
print("\nPredictions for New Samples:")
for pred in predictions:
    print(iris.target_names[pred])
```

```
Predictions for New Samples:
setosa
versicolor
virginica
```

```
Start coding or generate with AI.
```